

abstractizare

Șapte exerciții — `abstract` pentru familie, `interface` pentru capacități · TypeScript · Andreea

Unde lucrezi: `vanilla-ts/src/exercitii/` și `vanilla-ts/src/fise/`, peste ce ai făcut la fișele de moștenire și polimorfism.

Nou, ai voie: `abstract` pe clasă, pe metodă și pe câmp, `interface`, `implements`, `toString()` și `toJson()`.

Încă NU folosim: `instanceof`, `as`, `any`, generice proprii, `static`, `enum`.

Cum se lucrează: fiecare exercițiu îți cere să **construiești** ceva. Aici compilatorul e cel care verifică cel mai mult: la fiecare exercițiu rulezi `npx tsc --noEmit` și citești ce spune, înainte de `npx tsx`.

Partea 1 — clasa de sus nu se mai poate construi

1. Verificarea nu mai minte

nivel 1

Fișierul `05-verificari`. `Verificare.trece` întoarce `true` doar ca să compileze; nimeni nu vrea rezultatul ăla. Fă clasa **abstractă** și metoda **abstractă**.

Apoi scrie **Contine** (primește un fragment, trece dacă valoarea îl conține) intenționat pe jumătate: doar constructorul. Rulează `tsc`, notează mesajul, completează până trece.

Gata când: o linie `new Verificare("x")` scrisă în `main` e refuzată de `tsc` (o ștergi după); `Contine` intră în array și apare în ieșire; ieșirea veche e identică.

De reținut: un corp care există doar ca să treacă compilatorul e semnul metodei abstracte. Acum compilatorul nu te lasă să termini pe jumătate.

2. `text()` scris o singură dată

nivel 1

Fișierul `03-raport`. Acum fiecare subclasă rescrie `text()` cu `super.text()` plus ceva. Reorganizează: `Raport` devine abstractă, iar `text()` se compune într-un singur loc, în bază, din: `antet()`, titlul, liniile, `subsol()`. `antet()` și `subsol()` sunt abstracte.

`RaportCuData` dă doar antetul, `RaportSemnat` doar subsolul. Decizie: ce întoarce fiecare pentru partea pe care n-o are, și cum eviți o linie goală în ieșire? Scrie într-un rând.

Rularea veche care făcea `new Raport(...)` nu mai compilează. Decide ce pui în loc.

Gata când: `text()` apare într-un singur loc în fișier; ieșirea celor două subclase e identică cu cea de dinainte; `RaportCompus` merge neatins.

De reținut: metoda scrisă din bază cheamă metode nescrise încă. Baza fixează ordinea bucăților; subclasa dă bucata.

3. Regula rândului, ca metodă abstractă

mai greu

Fișierul `01-set-de-date`. `SetDeDateCuCheie` și `SetDeDateStrict` rescriu amândouă `adauga`. Reorganizează: `SetDeDate` devine abstractă, iar `adauga` rămâne scrisă **o singură dată**, în bază: numără valorile, cheamă `verificaRand(rand): void` (abstractă, aruncă dacă e ceva de reproșat), apoi adaugă. Cele două subclase nu mai au `adauga`; dau doar `verificaRand`.

Decizie: ai nevoie și de un set fără nicio regulă în plus. Ce clasă construiești pentru el?

Gata când: `adauga` apare într-un singur loc; cele trei mesaje de eroare apar ca înainte, fiecare la cazul lui; `numarRanduri()` nu crește la niciun refuz.

De reținut: baza fixează *ordinea pașilor* (numără, verifică, adaugă); subclasa umple un singur pas. Nimeni nu mai poate uita să numere coloanele.

Partea 2 — un contract peste clase fără rudă comună

4. Exportabil

mai greu

Folder nou: `src/fise/exportabil/main.ts`. Copiază acolo `Raport` cu subclasele, `SetDeDate` cu o subclasă și `Exportator` cu `ExportatorCsv`.

Un raport și un set de date n-au nicio rudă comună, dar amândouă se pot da unui exportator. Scrie `interface Exportabil` cu o singură metodă, `caRanduri(): string[][]`, și fă ambele ierarhii s-o implementeze: setul își dă rândurile, raportul dă fiecare linie ca un rând cu o singură valoare.

Scrie funcția `exporta(lucruri: Exportabil[], e: Exportator): void`, care afișează pentru fiecare numele fișierului și conținutul convertit. În `main`: o singură listă cu un `RaportSemnat` și un set.

Gata când: lista e declarată o singură dată și ține obiecte din ambele ierarhii; CSV-ul setului are câte un rând pe linie, al raportului câte o linie pe rând; în funcție nu apare `instanceof`.

De reținut: cu `extends` n-ai fi putut face lista asta. Un set de date nu este un raport; doar *poate* același lucru.

5. Contractul scris de altcineva

mai greu

Fișierul `03-raport` și `01-set-de-date`. Limbajul are deja două contracte pe care le respectă orice obiect care vrea: șablonul ``${obiect}`` cheamă `toString()`, iar `JSON.stringify(obiect)` cheamă `toJSON()` dacă există.

Dă-i lui `Raport` un `toString()` care întoarce `text()`. Dă-i lui `SetDeDate` un `toJSON()` care întoarce doar `{ coloane, numarRanduri }`, nu rândurile.

Înainte să le scrii, rulează `console.log(`${raport}`)` și `console.log(JSON.stringify(set))` și notează ce iese. Apoi scrie-le și rulează din nou.

Gata când: șablonul afișează textul raportului, nu `[object Object]`; JSON-ul setului nu conține nicio valoare din rânduri; iar tu n-ai chemat explicit nici `toString`, nici `toJSON`.

De reținut: aici n-ai inventat tu contractul. L-ai respectat pe al limbajului ca să-i poți folosi codul. Așa lucrezi cu orice bibliotecă, Playwright inclus.

6. Un intrus în listă

provocare

Folderul `src/fise/exportabil/`. Scrie **Notita**, cu titlu și text, care nu e nici raport, nici set de date, și fă-o `Exportabil`. Adaug-o în lista de la exercițiul 4.

Gata când: ies trei exporturi, iar `git diff` arată că funcția `exporta` n-a fost atinsă.

De reținut: asta e testul adevărat al unei interfețe: un tip la care nu te-ai gândit când ai scris-o întră fără să schimbi nimic.

7. Verificare : clasă abstractă sau interfață?

provocare

Fișierul `05-verificari`. Ai acum amândouă uneltele. Răspunde în scris, în cinci-șase rânduri: `Verificare` ar trebui să rămână clasă abstractă sau să devină interfață?

Pornește de la două întrebări concrete:

- Ce stare au în comun toate verificările (`nume`)? Cine ar ține-o dacă `Verificare` ar fi interfață?
- Există cod scris o singură dată în bază și folosit de toate subclasele? Ce s-ar întâmpla cu el?

Apoi implementează varianta aleasă și rulează tot fișierul, inclusiv `Negata` și `careNuTrec`.

Gata când: programul afișează la fel ca înainte, iar motivarea se potrivește cu ce a trebuit să muți sau să dublezi în cod.

De reținut: cele două nu se exclud. Cea mai obișnuită soluție e o clasă abstractă plus interfețe: familia vine din moștenire, capacitățile din contracte.